2026.1 - 2026.3 Kaggle

kaggle

Pig Posture Recognition 대회 보고서

참가자 : 문병선, 강지호, 신창용

목 차

1. Pig Posture Recognition 개요.....3

2. 이론적 배경.....5

3. 방법론

3.1 YOLO v11 Model을 이용한 방법론.....9

3.1.1 YOLO v11 Model 학습 데이터 생성.....9

3.1.2 YOLO v11 Model 훈련.....10

3.1.3 YOLO v11 Model 추론 및 평가.....12

3.2 EfficientNet_V2 Model을 이용한 방법론.....13

3.2.1 EfficientNet_V2 Model 학습 데이터 생성.....13

3.2.2 EfficientNet_V2 Model 훈련.....14

3.2.3 EfficientNet_V2 Model 추론 및 평가.....14

3.3 ConvNeXt Model을 이용한 방법론.....15

3.3.1 ConvNeXt Model 학습 데이터 생성.....15

3.3.2 ConvNeXt Model 훈련.....16

3.3.3 ConvNeXt Model 평가.....17

4. 결론.....19

1. Pig Posture Recognition 개요

Pig Posture Recognition

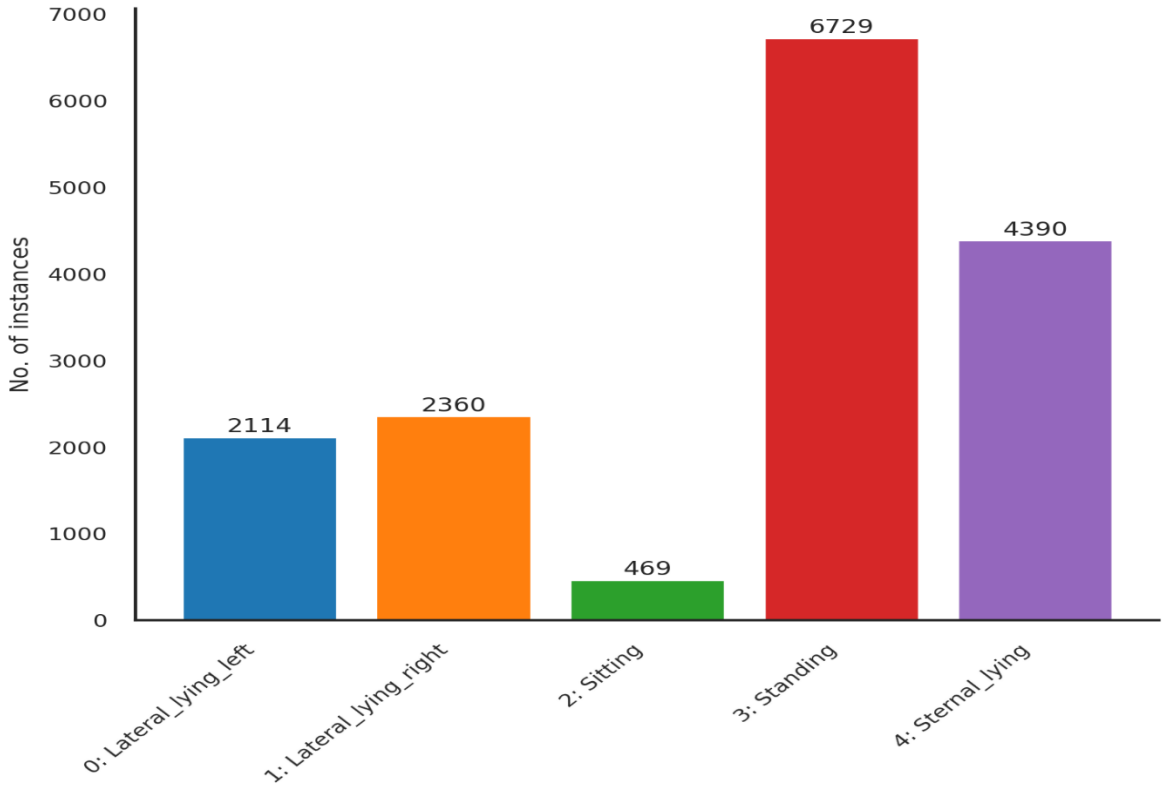
A Computer Vision Challenge in Precision Livestock Farming (PLF)

<Pig Posture Recognition >

실제 농장 환경에서 돼지의 자세를 정확하게 감지하는 것은 컴퓨터 비전 분야에서 여전히 어려운 과제이다. 통제된 실험실 환경과 달리, 상업용 축사는 복잡한 배경, 잦은 가림 현상, 가변적인 조명 조건, 그리고 다양한 카메라 시점 등 여러 가지 특징을 가지고 있다. 또한, 돼지의 자세는 미묘한 시각적 차이를 보이기 때문에 2D 이미지에서 자세를 구분하는 것이 특히 어렵다.

본 대회는 실제 농장 환경에서 딥러닝 모델의 개발 및 평가를 장려함으로써 돼지 자세 인식 기술을 발전시키는 것을 목표로 한다. 데이터셋은 미시간 주립대학교(MSU) 양돈 연구센터에서 수집한 3,090장의 이미지와 22,934개의 주석이 달린 인스턴스로 구성된다. 이미지는 8대의 오버헤드 카메라를 사용하여 촬영되었으며, 다양한 돼지 품종(순종 요크셔, 이국 교배종, 메이산)과 생산 배치를 포함한다.

데이터셋은 무작위로 훈련 및 테스트 하위 집합으로 나뉘며, 이미지의 70%(16,062개의 인스턴스를 포함하는 2,164개의 이미지)는 훈련에 할당되고 나머지 30%는 유사한 클래스 분포를 따라 테스트에 사용된다.



<훈련 데이터셋 통계>

각 이미지에는 우리 안에 3~10마리의 돼지가 있습니다. 데이터셋은 모든 돼지 개체에 대해 학습 셋과 테스트셋 모두에서 축 정렬된 경계 상자(xmin, ymin, w, h)를 제공한다. 단, 자세 레이블은 학습 세트에서만 제공된다. 참가자는 테스트셋에서 감지된 각 돼지 개체의 자세를 예측해야 한다. 각 돼지의 자세를 다음 다섯 가지 범주 중 하나로 분류하는 것이 과제이다.

- 왼쪽으로 누워있는 자세
- 오른쪽으로 누워있는 자세
- 좌석
- 서 있는
- 흉골이 누워있는

인스턴스 수준 이미지 분류, 표현 학습 및 모델 일반화 능력을 시험해 볼 뿐만 아니라, 기계 학습, 센서 기술 및 동물 과학이 융합되어 더욱 스마트하고 지속 가능한 축산 시스템을 구현하는 새로운 학제 간 분야인 정밀 축산(Precision Livestock Farming, PLF) 연구에도 기여하고 궁극적인 목표는 다양한 카메라 장치, 시점, 돼지 개체군 및 환경 조건에서 일관된 성능을 보이는 견고하고 일반화 가능한 자세 인식 모델을 구축하는 것이다.

대회에서는 모델 성능을 평가하기 위해 각 자세 유형별로 F1 점수를 독립적으로 계산한 후, 모든 유형에 빈도와 관계없이 동일한 가중치를 부여하여 평균을 낸다. 따라서 불균형 데이터셋에 특히 적합하다.

$$F1 = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

2. 이론적 배경

본 연구는 돼지 자세 분류 문제를 해결하기 위해 이미지 분류(Classification) 모델을 구축하고, 실제 Kaggle 환경에서 성능을 높이기 위한 전처리 및 학습/추론 전략을 적용하였다. 특히 입력 이미지에서 자세와 무관한 배경이 크게 포함될 수 있어, 객체 중심 정보(돼지 영역)에 집중하도록 Bounding Box 기반 ROI 전처리를 수행하였다.

2.1 YOLO 란?

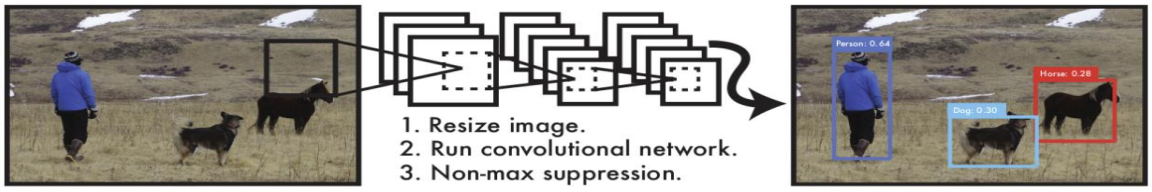


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448 × 448, (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

< The YOLO Detection System >

YOLO(You Only Look Once)는 실시간 객체 탐지를 위한 딥러닝 기반 기법으로, 이미지 내 객체의 위치와 클래스를 한 번의 신경망 연산으로 예측하는 방식이다. 기존 객체 탐 지 기법이 다단계 과정을 거치는 것과 달리, YOLO는 단일 신경망을 활용한 종단 간 (end-to-end) 예측 방식을 사용하여 빠른 속도로 객체를 탐지할 수 있다.

2.1.1 YOLO v11 구조 및 특징

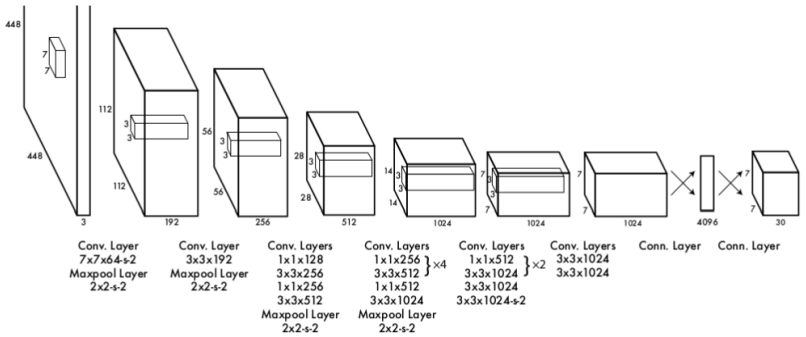
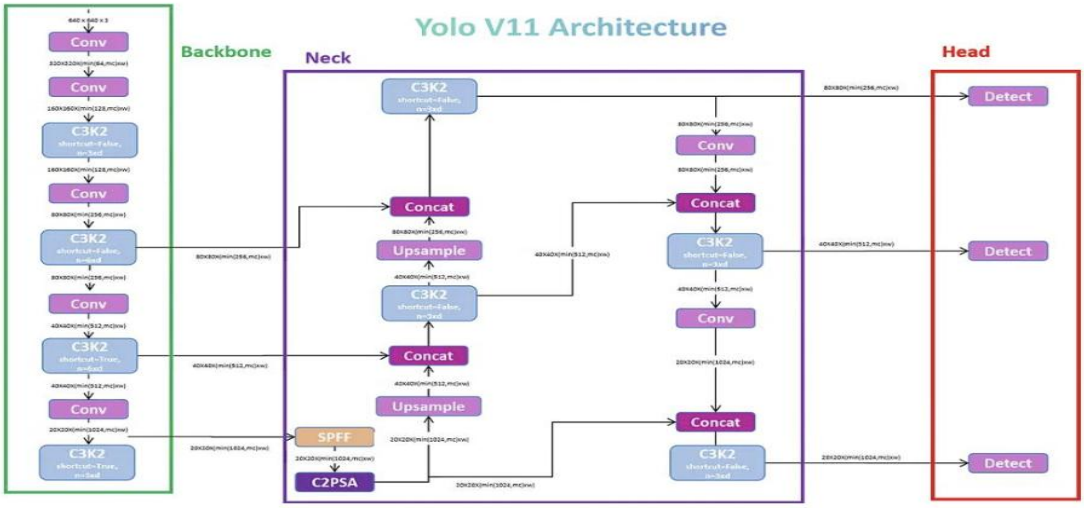


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1 × 1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224 × 224 input image) and then double the resolution for detection.



< YOLO v11 Model Architecture >

YOLO v11은 기존 YOLO 시리즈의 최신 모델로, 객체 탐지 성능을 향상시키기 위해 다중 스케일 학습, 향상된 백본 네트워크, 더욱 정밀한 바운딩 박스 예측 기법 등을 적용하였 다. 특히, YOLO v11은 기존 모델 대비 소형 객체 탐지와 가려진 객체 탐지 성능이 향상 되었으며, 자율주행 및 실시간 영상 처리 시스템에서 활용될 가능성이 크다.

2.2 EfficientNet V2란?

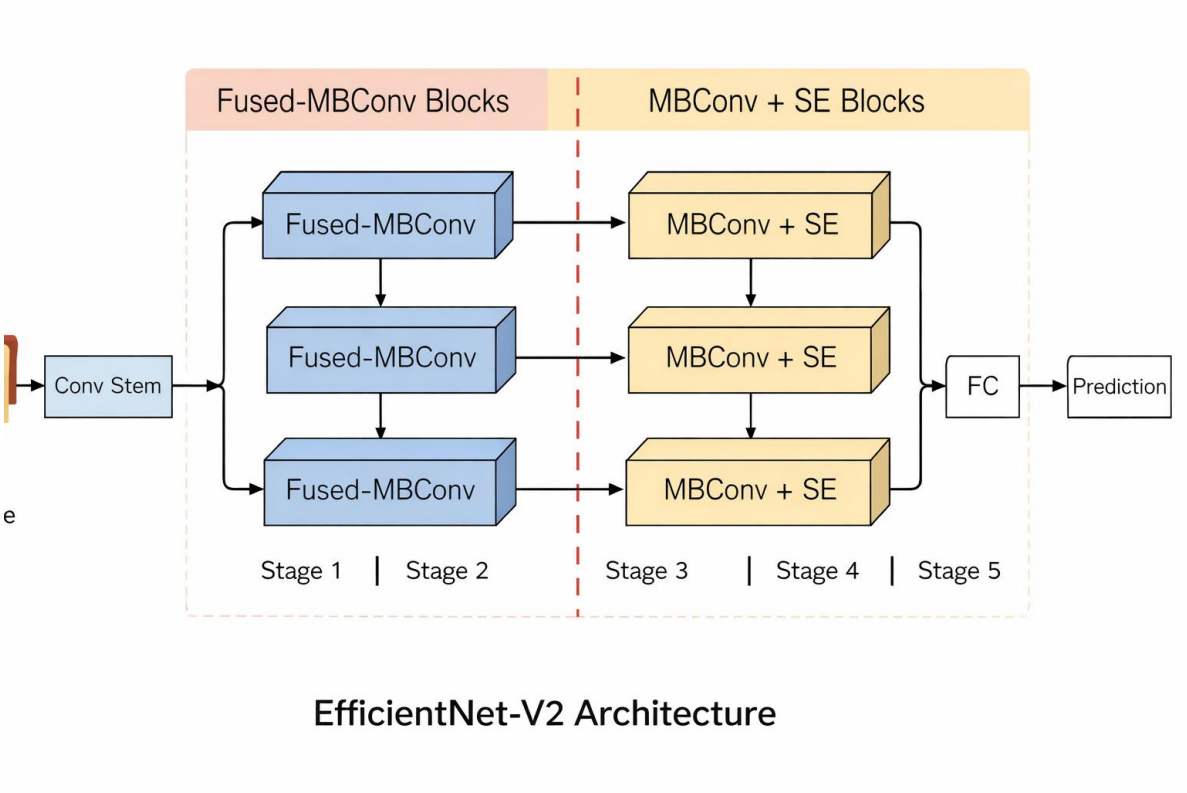
EfficientNetV2는 Google에서 제안된 CNN 기반 이미지 분류 모델로, 기존 EfficientNet 구조를 개선하여 더 빠른 학습 속도와 높은 정확도를 동시에 달성하도록 설계된 모델이다.

2.2.1 EfficientNet V2 구조 및 특징

EfficientNetV2는 기존 EfficientNet의 MBConv 구조에 더해 Fused-MBConv 구조를 도입하여 초기 학습 단계에서의 연산 효율성을 개선하였다. 이를 통해 대규모 이미지 데이터에서도 빠르게 학습할 수 있으며, 높은 성능을 유지할 수 있다.

본 연구에서는 ImageNet 데이터로 사전 학습된(pretrained) EfficientNetV2-M 모델을 기반으로 전이학습(Transfer Learning)을 수행하였다. 마지막 Fully Connected Layer를 돼지 자세 분류 문제에 맞게 5개의 클래스 출력으로 수정하여 모델을 학습하였다.

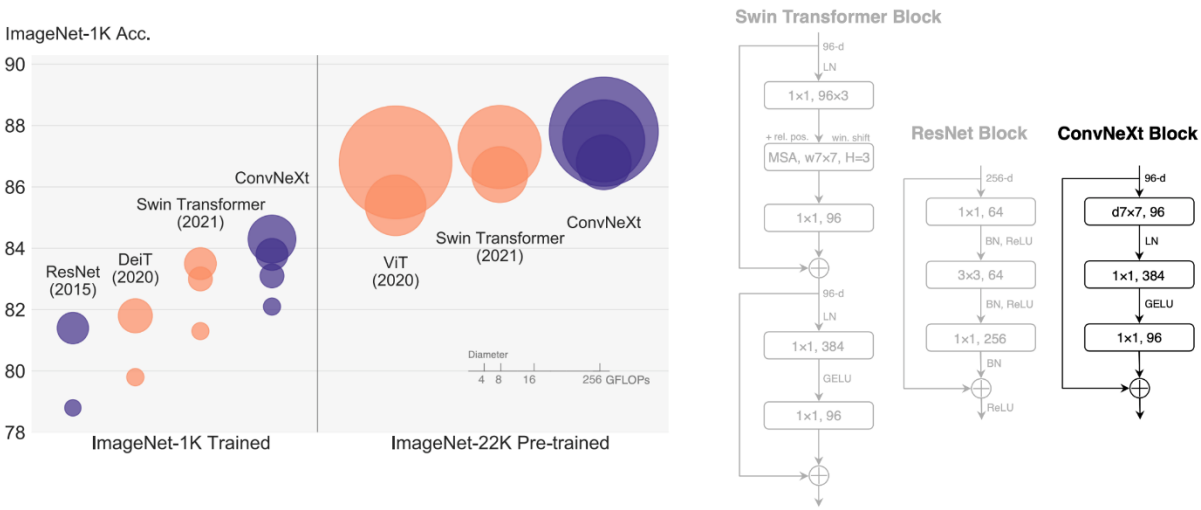
이와 같은 방식으로 구축된 모델은 입력된 이미지에서 돼지의 특징을 효과적으로 추출하여 다양한 자세를 정확하게 분류할 수 있다.



< EfficientNet-V2 Model Architecture >

2.3 ConvNeXt 란?

ConvNeXt는 2022년 Meta AI에서 제안한 합성곱 신경망(Convolutional Neural Network, CNN) 기반의 이미지 분류 모델이다. 해당 모델은 *A ConvNet for the 2020s* 논문에서 발표되었으며, 기존 ResNet 계열 CNN을 현대화하여 Transformer 기반 모델과 경쟁 가능한 성능을 달성하는 것을 목표로 설계되었다.

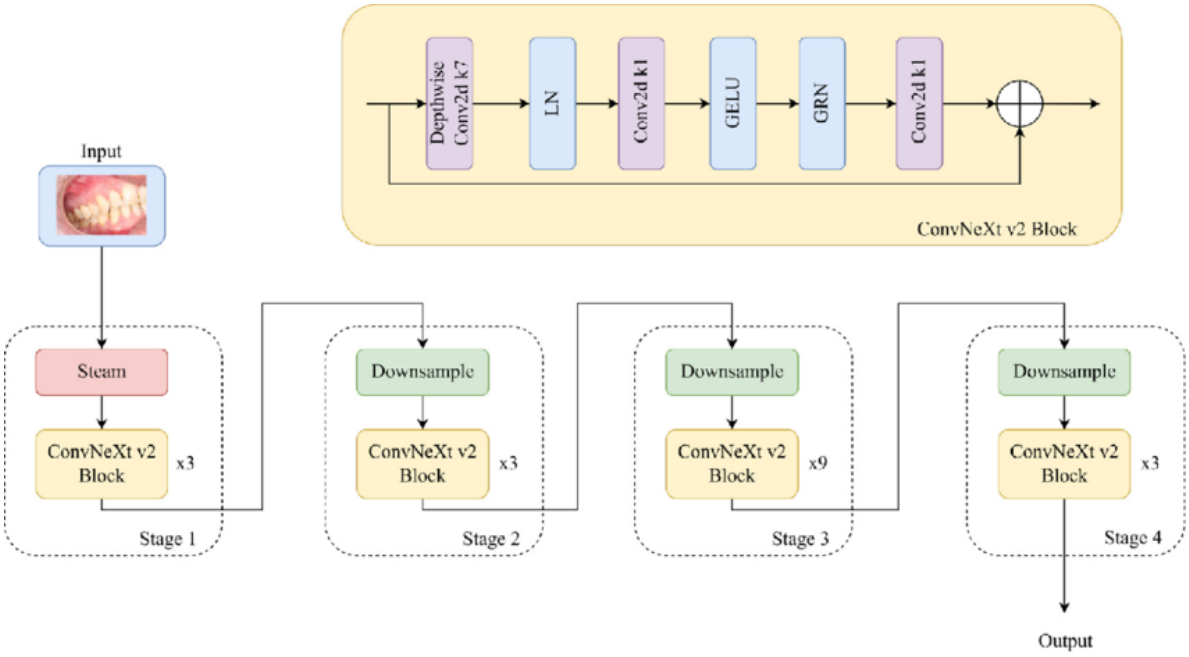


< ConvNeXt 모델 성능 비교 >

최근 Vision Transformer(ViT) 계열 모델이 이미지 분류 분야에서 높은 성능을 보이면서 CNN의 한계가 제기되었으나, ConvNeXt는 CNN 구조를 Transformer 설계 철학에 맞게 재구성함으로써 CNN의 구조적 장점을 유지하면서도 최신 성능을 달성할 수 있음을 보였다.

특히 livestock posture recognition과 같은 데이터셋에서는 배경 texture와 자세 형태 정보가 동시에 중요하기 때문에 local feature extraction이 강한 CNN 기반 모델이 Vision Transformer보다 안정적인 성능을 보이는 경우가 많다.

2.3.1 ConvNeXt 구조 및 특징



< ConvNeXt Model Architecture >

ConvNeXt의 기본 블록은 기존 ResNet 블록과 유사한 잔차 연결(Residual Connection)을 유지하면서도 내부 구조를 Transformer와 유사한 형태로 변경하였다. 기존 CNN에서 주로 사용되던 Batch Normalization 대신 Layer Normalization을 적용하였으며, 이는 배치 크기에 대한 의존성을 줄이고 Transformer와 유사한 학습 특성을 갖도록 한다.

ConvNeXt block은 depthwise convolution → LayerNorm → pointwise convolution 구조로 구성되며 large kernel (7×7)을 사용하여 CNN의 receptive field를 확장한다. 이는Transformer의 global receptive field 특성을 CNN 구조에서 근사하려는 설계이다.

또한 입력 단계에서 기존 CNN의 7×7 Convolution + MaxPooling 대신, 4×4 Stride 4 Convolution을 사용하는 Patchify Stem을 도입하였다. 이는 ViT에서 사용되는 패치 분할 방식과 유사한 접근으로, 초기 단계에서 해상도를 빠르게 축소하고 특징 표현을 효율적으로 구성한다.

3. 방법론

본 연구에서는 돼지 자세 분류(Pig Posture Recognition)를 위해 YOLO v11 기반 이미지 분류(Classification) 모델을 사용하였다 특히 원본 이미지에는 배경 영역이 크게 포함되어 있어 자세 분류에 불필요한 특징이 학습될 가능성이 있으므로, 제공되는 bbox를 활용하여 객체 중심의 학습 데이터를 구성하였다.

3.1 YOLO v11 Classification Model을 이용한 방법론

본 방법론에서 사용한 모델은 Ultralytics 기반 YOLO11m-cls이며, 중간 규모(medium)의 모델 용량을 사용하여 표현력을 확보하면서도 Kaggle 환경에서 학습이 가능하도록 설정하였다. 학습 과정에서는 사전 학습 가중치를 기반으로 전이학습을 수행하고, 과적합 방지를 위해 dropout 및 weight decay를 적용하였다.

또한 좌/우 자세 클래스(Lateral_left / Lateral_right)와 같은 방향성 레이블의 혼동을 막기 위해 fliplr을 0으로 고정하였다. 이는 단순 좌우반전 증강이 레이블 의미를 바꿀 수 있는 문제를 방지하기 위한 설정이다.

3.1.1 YOLO v11 Classification Model 학습 데이터 생성

YOLO 분류 모델 학습을 위해, 제공된 train.csv의 bbox 정보를 활용하여 row_id 단위로 학습 이미지를 오프라인 생성하였다. 생성 과정은 다음과 같다.

1) 데이터 수집 및 전처리(ROI crop)

각 이미지에 대해 bbox 문자열 [x, y, w, h]를 파싱한 뒤, 해당 영역만 crop한다. 이때 bbox 주변 정보를 일부 포함시키기 위해 PAD=0.10을 적용하여 bbox 크기의 10%만큼 확장된 영역을 사용하였다.
이 과정은 bbox 경계가 완벽하지 않거나 자세 판단에 주변 컨텍스트가 필요한 경우에도 성능 저하를 줄이는 데 목적이 있다.

2) Letterbox(평균색 padding) 기반 정사각 입력 생성

crop된 ROI는 원본 비율이 제각각이므로 모델 입력을 위해 정사각 이미지로 변환이 필요하다. 본 연구에서는 단순 resize로 인한 왜곡을 피하기 위해 비율 유지(letterbox) + padding을 적용하였다.

특히 padding 색상을 검정(0)으로 채울 경우 패딩 영역이 강한 인공 패턴으로 남아 모델이 "패딩 지문"을 학습할 수 있으므로, padding 색상으로 해당 crop 이미지의 평균색을 사용하였다. 이는 패딩으로 인해 생기는 분포 차이를 완화하고 일반화 성능을 높이기 위한 선택이다.

3) 클래스 불균형 완화: Sitting 약한 oversampling + 약한 offline augmentation

훈련 데이터에서 Sitting 클래스의 상대적 약세를 보완하기 위해, Sitting 클래스만 2.0배

(OS_SITTING_MULT=2.0)로 oversampling하였다. 다만 과한 oversampling은 같은 이미지 반복으로 과적합을 유도할 수 있으므로 "약한 증식" 수준으로 제한하였다.

또한 Sitting 클래스에 한하여 oversampling된 샘플 중 일부(AUG_SITTING_PROB=0.30)에 대해 약한 오프라인 증강을 적용하였다.

- 약한 밝기/대비 변형(조명 변화 대응)
- 작은 blur(초점 변화/노이즈 대응)
- 작은 Gaussian noise(센서 노이즈 대응)
- 작은 random erasing(특정 부위 과의존 완화)

이 전략은 "데이터 수 자체를 크게 늘리기"보다, 약한 다양성 확보로 Sitting의 일반화 성능을 보완하는 목적에 가깝다.

4) 데이터셋 폴더 구조 생성

YOLO 분류 학습 형식에 맞게 아래 구조로 이미지를 저장한다.

- {out_root}/train/{class_name}/*.jpg
- {out_root}/val/{class_name}/*.jpg

즉, class_name 디렉토리 기반으로 자동 레이블링이 되도록 구성하여 학습 파이프라인을 단순화하였다.

3.1.2 YOLO v11 Classification Model 훈련

본 연구는 과적합을 줄이고 안정적인 수렴을 유도하기 위해 아래 학습 전략을 적용하였다.

1) K-Fold 기반 학습

데이터 분할은 StratifiedKFold(n_splits=5)를 사용하여 클래스 분포가 각 fold에 균일하게 반영되도록 하였다. 이 중 fold 0, 1을 학습에 사용하였으며(필요 시 2~3 fold로 확장 가능), fold별 best weight를 저장하여 앙상블에 활용하였다.

2) 학습 하이퍼파라미터 및 규제(Regularization)

- epochs: 35
- batch: 8 (메모리 제한 고려)
- lr0: 6e-4
- optimizer: AdamW
- cos_lr: True (코사인 스케줄로 후반 안정화)

- dropout: 0.05
- weight_decay: 7e-4
- warmup_epochs: 2.0
- patience: 8 (조기 종료)

또한 YOLO 내부 증강은 과도하면 자세 분류에서 오히려 손해가 될 수 있어, 강도를 낮춘 형태로 적용하였다.

- mixup = 0.02
- cutmix = 0.02
- auto_augment = randaugment
- erasing = 0.20 (과한 가림으로 자세 특징 손실 위험 완화)

3) 좌/우 레이블 보호를 위한 flip 비활성화

좌우가 의미를 가지는 레이블이 존재하므로, fliplr/flipud는 다음과 같이 비활성화하였다.

- fliplr = 0.0
- flipud = 0.0

이는 단순 반전 증강이 정답 레이블을 바꾸는 "라벨 노이즈"를 만들 수 있기 때문이다.

4) OOM(Out Of Memory) 대비 fallback 전략

학습 해상도는 imgsz=448을 기본으로 사용하되, GPU 메모리 부족(OOM) 발생 시 imgsz=384로 자동 fallback하도록 설계하였다.

3.1.3 YOLO v11 Classification Model 추론 및 평가

훈련이 완료된 후, 테스트 데이터에 대해 다음 전략으로 추론을 수행하였다.

1) 테스트 입력 전처리

훈련과 동일하게

- bbox crop(+PAD=0.10)
- meanpad letterbox(imgsz 동일)

을 적용하여 입력 분포를 일치시켰다.

2) 5-crop TTA(Test Time Augmentation)

각 테스트 이미지에 대해 5개의 crop view를 생성하였다.

- center crop
- top-left, top-right
- bottom-left, bottom-right

각 view에 대해 모델이 출력한 확률을 결합할 때 단순 평균이 아니라 log-probability 평균을 사용하였다. 이 방식은 매우 작은 확률값에서 수치적으로 안정적이며, 확률 곱에 가까운 결합 효과를 갖기 때문에 여러 view에서 일관된 클래스를 더 강하게 지지하도록 만든다.

3) Fold 앙상블(2~3 fold)

학습된 fold 모델들에 대해 동일한 TTA 추론을 수행한 뒤, fold별 score(log-prob)를 평균으로 최종 예측을 생성하였다.

- fold별 log-prob 평균
- fold 평균 후 softmax 적용
- argmax로 class_id 결정

이 과정은 단일 모델 예측의 분산을 낮추고, 특정 fold에서의 편향을 줄여 안정적인 제출 성능을 기대할 수 있다.

3.2 EfficientNet_V2 Model을 이용한 방법론

본 프로젝트는 사전학습된 합성곱 신경망 모델을 기반으로 이미지 분류기를 구축한 후, 단계적인 실험을 통해 성능을 개선하는 방식으로 진행되었다. 초기 베이스라인 모델에서 출발하여 학습 전략, 데이터 증강 기법, 정규화 기법을 순차적으로 적용하였으며, 각 단계에서의 성능 변화를 분석하였다.

성능 향상 과정은 크게 기본 모델 구축 단계, 학습 안정화 단계, 데이터 분포 확장 단계, 정규화 및 추론 최적화 단계로 구분할 수 있다.

3.2.1 EfficientNet_V2 Model의 학습 데이터 생성

초기 모델은 ImageNet 사전학습 가중치를 활용한 EfficientNet_v2_m 구조를 기반으로 구성하였

다. 데이터 전처리는 제공된 Bounding Box 정보를 활용하여 객체 중심으로 크롭한 뒤, 고정 해상도로 resize 및 정규화를 수행하였다. 기본 증강으로는 Random Horizontal Flip을 적용하였다.

1) Bounding Box Margin Randomization

객체를 정확히 크롭하는 대신, 일정 비율(약 5~15%)의 랜덤 마진을 추가하여 객체 주변 정보를 일부 포함하도록 하였다. 이를 통해 객체 중심 과적합을 완화하고 다양한 구도에 대한 학습을 유도하였다.

2) Color Jitter 적용

조명 변화 및 색상 편차에 대한 강건성을 확보하기 위해 brightness, contrast, saturation, hue를 무작위로 변형하는 Color Jitter를 적용하였다. 특히 색상 기반 증강은 모델이 조명 조건 변화에 민감할 수 있다는 점을 보완하기 위한 방법으로 사용되었다.

< Data augmentation example >

3.3.2 EfficientNet_V2 Model의 훈련

학습 설정은 다음과 같다.

- Loss Function: CrossEntropyLoss
- Optimizer: AdamW
- Learning Rate: 고정값 사용
- Epoch: 약 30~35
- 기본 증강: Random Horizontal Flip

부 클래스에서 과도한 확신(confidence)을 보이는 경향이 관찰되었다. 이를 통해 모델의 일반화 성능이 충분히 확보되지 않았음을 확인하였다.

초기 모델의 성능 한계를 분석한 결과, 학습 과정의 안정성이 성능을 제한하는 요인으로 판단되었다. 이에 따라 학습률 스케줄링과 혼합 정밀도 학습을 도입하였다.

1) CosineAnnealingLR 적용

학습 후반부로 갈수록 learning rate가 점진적으로 감소하도록 설정하였다. 이를 통해 학습 후반의 손실 진동을 완화하고 안정적인 수렴을 유도하였다.

2) Mixed Precision(AMP) 학습 적용

연산 효율을 개선하고 GPU 메모리 사용량을 감소시켰다. 이는 batch size 유지 및 학습 안정성 확보에 기여하였다.

이후 성능이 다시 정체됨에 따라, 모델의 과도한 확신을 완화하고 일반화 성능을 향상시키기 위한 정규화 전략을 도입하였다.

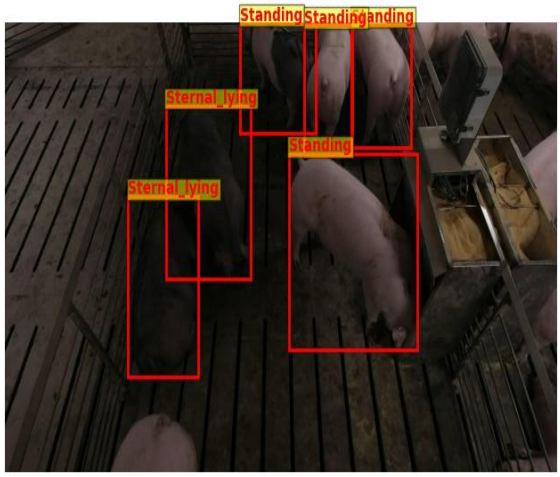
3) Label Smoothing

CrossEntropyLoss에 Label Smoothing(0.1)을 적용하여 정답 라벨을 완전한 one-hot 벡터가 아닌 완화된 확률 분포 형태로 변환하였다. 이는 모델이 특정 클래스에 과도하게 확신하는 현상을 줄이고, 과적합을 완화하는 데 기여하였다.

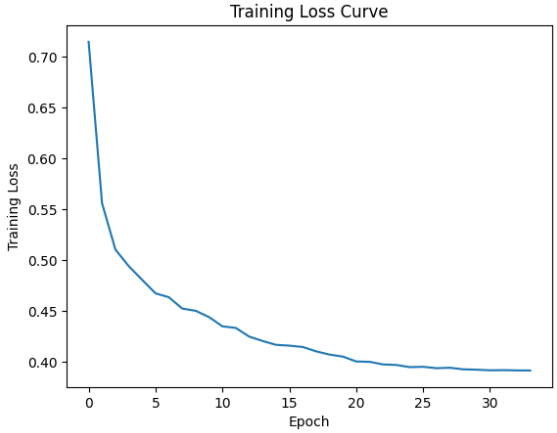
3.2.3 EfficientNet_V2 Model의 추론 및 평가

추론 단계에서는 원본 이미지와 좌우 반전 이미지를 함께 예측한 후 확률 평균을 계산하는 Test Time Augmentation(TTA)을 적용하였다. 적절한 TTA는 예측 안정성을 높이는 데 기여하였으나, 과도한 증강 조합은 오히려 성능 저하를 초래하였다.

또한 전체 실험 과정에서 원본 이미지와 증강된 이미지의 변화를 함께 확인하였으며, Bounding Box 예측 결과 이미지, 데이터 셋의 클래스 분포, Epoch 진행에 따른 Training Loss 변화 등을 바탕으로 모델의 동작 특성과 학습 경향을 분석하였다.



< BBox 예측 결과 이미지 >



< Epoch 진행에 따른 Training Loss 변화 >

3.3 ConvNeXt Model을 이용한 방법론

본 방법론에서는 ConvNeXt-Small을 backbone으로 사용하고, ImageNet-1K 사전학습 가중치를 로드한 뒤 최종 분류기 헤드를 본 과제의 클래스 수(5개)에 맞게 교체하여 전이학습을 수행하였다.

학습은 클래스 불균형을 완화하기 위해 WeightedRandomSampler를 사용했으며, AdamW와 CosineAnnealingLR 스케줄러로 최적화하였다. 과도한 확신을 줄이기 위해 label smoothing을 적용하고, 약한 증강(Affine/ColorJitter/CoarseDropout)과 MixUp·CutMix를 통해 일반화를 보강했다.

3.3.1 ConvNeXt Model의 학습 데이터 생성

ConvNeXt 분류 모델 학습을 위해 제공된 train.csv의 bbox 정보를 활용하여 각 row_id 단위로 입력 이미지를 생성하였다. 입력 생성 과정은 다음과 같다.

1) 데이터 수집 및 ROI 기반 crop

각 이미지에 대해 bbox 문자열 [x, y, w, h]를 파싱한 뒤 해당 영역을 중심으로 crop을 수행하였다. 이때 bbox 경계의 불완전성과 자세 판단에 필요한 주변 정보를 일부 포함하기 위해 pad_ratio=0.20을 적용하여 bbox 크기의 20%만큼 확장된 영역을 사용하였다. 이러한 컨텍스트 패딩은 객체 주변의 환경 정보가 자세 분류에 도움이 되는 경우 성능 저하를 줄이기 위한 목적을 가진다.

2) Letterbox 기반 정사각 입력 생성 (Mean Padding)

crop된 이미지는 원본 비율이 서로 다르므로 모델 입력을 위해 정사각 이미지로 변환하였다. 단순 resize는 객체 비율을 왜곡할 수 있으므로 letterbox 방식을 적용하여 비율을 유지한 채 정사각 캔버스를 생성하였다. 이때 padding 영역은 검정색(0)이 아닌 ImageNet mean 색상(PAD_COLOR)으로 채웠다. Zero padding은 이미지 경계에 인공적인 패턴을 만들어 CNN이 해당 영역을 특징으로 학습할 가능성이 있으므로, 평균 색 기반 padding을 통해 입력 분포의 변화를 최소화하였다. 이후 이미지는 IMG_SIZE=384로 리사이즈하고 ImageNet mean/std 기준 정규화를 적용하였다.

3) 데이터 누수 방지를 위한 그룹 기반 split

같은 원본 이미지에서 생성된 여러 bbox 샘플이 train과 validation에 동시에 포함될 경우 데이터 누수가 발생할 수 있다. 이를 방지하기 위해 image_id에서 확장자를 제거한 image_group 기준으로 샘플을 묶고, 각 그룹의 최빈 class를 기준으로 StratifiedShuffleSplit을 수행하여 단일 train/validation 분할을 구성하였다.

< Data augmentation example>

3.3.2 ConvNeXt Model 훈련

본 연구에서는 torchvision의 ConvNeXt-Small을 backbone으로 사용하고 ImageNet 사전학습 가중치를 기반으로 전이학습을 수행하였다. 최종 분류기 헤드는 본 과제의 클래스 수(5개)에 맞게 수정하였다.

1) 클래스 불균형 완화

훈련 데이터의 클래스 분포 차이를 완화하기 위해 WeightedRandomSampler를 적용하였다. 각 클래스의 샘플 수에 반비례하는 가중치를 부여하여 소수 클래스가 충분히 학습되도록 하였다.

2) 학습 하이퍼파라미터 및 최적화

학습 최적화는 AdamW optimizer를 사용하고 CosineAnnealingLR 스케줄러를 통해 학습률을 점진적으로 감소시켰다.

주요 하이퍼파라미터는 다음과 같다.

- epochs: 12
- batch size: 16
- learning rate: 2.5e-4
- weight decay: 0.02
- optimizer: AdamW
- scheduler: CosineAnnealingLR

또한 모델의 과도한 확신(confidence)을 완화하기 위해 label smoothing(0.02)을 적용하였다.

3) 데이터 증강 및 regularization

모델의 일반화 성능을 향상시키기 위해 다음과 같은 데이터 증강을 적용하였다.

- transform (scale, translation, rotation)
- ColorJitter (brightness, contrast 등)

- CoarseDropout

추가적으로 MixUp와 CutMix(alpha=0.2, 적용 확률 0.5)를 사용하여 서로 다른 이미지 간 특징을 혼합함으로써 경계 샘플에 대한 강건성을 높였다.

4) EMA 기반 학습 안정화

학습 과정에서 파라미터 변동을 줄이고 일반화 성능을 안정화하기 위해 EMA(Exponential Moving Average, decay=0.999)를 사용하였다. EMA weight는 매 iteration마다 업데이트되며 validation 평가 및 best 모델 저장은 EMA weight 기준으로 수행하였다.

3.3.3 ConvNeXt Model 추론 및 평가

훈련된 모델을 이용하여 테스트 데이터에 대해 다음과 같은 추론 전략을 적용하였다.

1) 테스트 입력 전처리

훈련 단계와 동일하게

- bbox crop + context padding (pad_ratio=0.20)
- mean padding 기반 letterbox
- IMG_SIZE=384 resize

를 적용하여 입력 분포의 일관성을 유지하였다.

2) Conditional TTA(Test-Time Augmentation)

모든 샘플에 TTA를 적용하면 계산 비용이 크게 증가하므로, 불확실 샘플에 대해서만 TTA를 적용하는 conditional TTA 전략을 사용하였다.

먼저 기본 입력에 대해 softmax 확률을 계산한 뒤 다음 조건 중 하나를 만족하면 불확실 샘플로 정의하였다.

- top-1 확률 < 0.78
- top1-top2 margin < 0.12

불확실 샘플에 대해서는 다음 과정을 수행하였다.

(1) Flip TTA

좌우 반전 이미지를 추가 추론한 뒤, 좌/우 자세 클래스의 의미가 뒤바뀌는 문제를 해결하기 위해 해당 클래스 확률을 swap한 후 원본 예측과 평균하였다.

(2) Pad-ratio TTA

flip 적용 이후에도 불확실한 경우 bbox 주변 컨텍스트가 달라질 때 예측이 안정화될 수 있다는

가정 하에 pad_ratio = 0.28, 0.36 두 가지 입력을 추가 생성하여 예측 확률을 평균하였다.

3) TTA 효과 분석

조건부 TTA의 실제 효과를 분석하기 위해 다음 통계를 기록하였다.

- base 추론에서 불확실로 분류된 샘플 비율
- flip 이후에도 불확실로 남은 샘플 비율
- pad-ratio TTA 적용 후 argmax 클래스 변화 비율

실험 결과 전체 샘플 중 약 0.15%만이 불확실 샘플로 분류되었으며, pad-ratio TTA가 적용된 샘플 중 약 50%에서 최종 argmax가 변화하였다. 이는 전체 데이터 기준 영향은 제한적이지만, 결정 경계 근처 샘플에서 예측 보정 효과가 있음을 보여준다.

4. 결론

4.1 YOLO v11 Classification Model의 대회 결과

본 연구에서는 YOLO11m 기반 분류 모델을 사용하여 돼지 자세 인식 성능을 개선하였다. 초기 baseline 모델은 단순 bbox crop 기반 학습으로 Public LB 기준 0.723의 F1-score를 기록하였다. 이후 전처리와 학습 전략을 단계적으로 개선하면서 성능을 향상시켰다.

Setting	**Public LB**
YOLOv11s baseline	0.723
+ Image size 증가	0.820
+ TTA (Multi-crop inference)	0.857
+ Pseudo-labeling	0.878
+ Hyperparameter tuning	0.916

4.2 EfficientNet V2 model의 대회 결과

EfficientNet V2-s의 경우 기본적으로 학습에 오랜 시간이 걸리지 않는 모델로서, 경량화에 치중하기보다는 시간이 늘어나더라도 정확도를 높이는 데 중점을 두고 학습을 진행하였다. 이를 위해 TTA와 random margin 같은 기법을 적용하였으며, 데이터 증강의 강도 조절과 epoch, learning rate 조절을 통해 기존 0.914점에서 0.956점까지 성능을 향상시켰다.

또한 초기 베이스라인 모델에서 시작하여 학습 안정화, 데이터 증강 강화, 정규화 전략 도입을 거치며 단계적으로 성능이 향상되었고, 특히 색상 기반 데이터 증강과 Label Smoothing이 주요 상승 요인으로 작용하였다. 본 실험을 통해 모델 구조 변경보다 데이터 분포 확장과 학습 전략 개선이 일반화 성능 향상에 더 중요한 역할을 한다는 점을 확인하였다.

Setting	**Public LB**
EfficientNet V2 baseline	0.914
+ Bbox margin 추가(20%)	0.924
+ TTA	0.941
+ Bbox random margin(5~15%)	0.950
+ Hyperparameter tuning	**0.956**

4.3 ConvNeXt Model의 대회 결과

본 대회에서는 ConvNeXt 기반 모델에 mean padding, EMA, conditional TTA를 적용하여 돼지 자세 인식 성능을 개선하였다. 초기 baseline 모델의 Public leaderboard 점수는 0.915였으나, 제안한 방법을 적용한 최종 모델은 0.948을 기록하였다. 특히 조건부 TTA는 전체 샘플 중 소수의 불확실 샘플에만 적용되면서도 결정 경계 근처 예측을 보정하는 효과를 보여 성능 향상에 기여하였다.

Setting	**Public LB**
Zero padding baseline	0.915
Mean padding	0.925
+ Conditional TTA	0.930
+ EMA	0.944
+ Parameter adjustment	0.948

4.4 결론

본 대회에서 사용한 세 가지 모델은 구조적으로는 서로 차이가 있었으나, 성능 향상에 기여한 핵심 요소는 공통적으로 나타났다.

첫째, Bounding Box 기반 ROI crop을 통해 자세와 무관한 배경 정보를 줄이고 객체 중심의 입력을 구성함으로써 자세 분류에 필요한 특징에 집중할 수 있었다.

둘째, Color Jitter, random margin, MixUp, CutMix 등 다양한 데이터 증강 기법을 적용하여 실제 농장 환경에서 발생할 수 있는 조명 변화, 시점 변화, 가림 및 경계 샘플에 대한 일반화 성능을 높였다.

셋째, AdamW와 cosine 계열 learning rate scheduling, AMP, EMA 등 학습 안정화 전략을 통해 수렴을 안정적으로 유도하고 과적합을 완화하였다.

넷째, Label Smoothing과 TTA, 앙상블 같은 추론 보정 기법을 통해 모델의 과도한 확신을 줄이고 최종 예측의 일관성을 높였다. 따라서 본 실험을 통해, 세 모델의 성능 향상은 단순한 모델 구조 차이보다도 입력 전처리, 데이터 분포 확장, 학습 안정화, 추론 보완의 네 요소를 얼마나 체계적으로 설계했는가에 더 크게 좌우된다는 점을 확인할 수 있었다.